

计算机视觉课程课后作业六

建智 2201 谢天赐 20221082044

6.1 通过编程完成任务

问题(1):

使用了 `with` 语句打开 `3DS.txt`, 并用 `readlines()` 方法读取文件的所有行, 然后使用 `strip()` 方法去除每行的空白符与换行符。其中, 根据文本内容格式, 使用 `split(' ', ')` 方法按逗号分割每行得到点的 ID 与坐标。主要代码如图 1 所示。

图 1

输出内容如图 2、3 所示，图 2 为 points 数据，格式为（‘序号’，x，y，z）图 3 为 faces 数据，格式为（‘序号’，[点 1，点 2，点 3，点 4]）：

```

[('0000', -100.0, 0.0, 50.0), ('0001', -48.0385, -30.0, 50.0), ('0002', -48.0385, 30.0, 50.0),
 ('0003', -100.0, 60.0, 50.0), ('0004', -151.9615, 30.0, 50.0), ('0005', -151.9615, -30.0,
 50.0), ('0006', -100.0, -60.0, 50.0), ('0007', -48.0385, -30.0, 50.0), ('0008', -100.0, 0.0,
-50.0), ('0009', -48.0385, 30.0, -50.0), ('0010', -48.0385, 30.0, -50.0), ('0011', -100.0,
-60.0, -50.0), ('0012', -151.9615, 30.0, -50.0), ('0013', -151.9615, -30.0, -50.0), ('0014',
-100.0, -60.0, -50.0), ('0015', -48.0385, -30.0, -50.0), ('0016', 100.0, 0.0, 50.0), ('0017',
151.9615, -30.0, 50.0), ('0018', 151.9615, 30.0, 50.0), ('0019', 100.0, 60.0, 50.0),
 ('0020', 48.0385, 30.0, 50.0), ('0021', 48.0385, -30.0, 50.0), ('0022', 100.0, -60.0, 50.0),
 ('0023', 151.9615, -30.0, 50.0), ('0024', 100.0, 0.0, -50.0), ('0025', 151.9615, -30.0,
-50.0), ('0026', 151.9615, 30.0, -50.0), ('0027', 100.0, 60.0, -50.0), ('0028', 48.0385,
30.0, -50.0), ('0029', 48.0385, -30.0, -50.0), ('0030', 100.0, -60.0, -50.0), ('0031',
151.9615, -30.0, -50.0), ('0032', 0.0, 0.0, 125.0), ('0033', 1.4142, -1.4142, 125.0),
 ('0034', -1.4142, -1.4142, 125.0), ('0035', -1.4142, 1.4142, 125.0), ('0036', -1.4142, -1.4142,
125.0), ('0037', 1.4142, -1.4142, 125.0), ('0038', 0.0, 0.0, 0.0), ('0039', 1.4142, -1.4142,
125.0), ('0040', 1.4142, 1.4142, 0.0), ('0041', -1.4142, 1.4142, 0.0), ('0042', -1.4142,
-1.4142, 0.0), ('0043', 1.4142, -1.4142, 0.0), ('0044', 0.0, 0.0, 135.0), ('0045', 0.0, 0.0,
135.0), ('0046', 0.0, 0.0, 135.0), ('0047', 0.0, 0.0, 135.0), ('0048', 0.0, 0.0, 135.0),
 ('0049', 0.0, 0.0, 135.0), ('0050', 0.0, 0.0, 135.0), ('0051', 0.0, 0.0, 135.0), ('0052',
0.0, 0.0, 135.0), ('0053', 0.0, 0.0, 135.0), ('0054', 0.0, 0.0, 135.0), ('0055', 0.0, 0.0,
0.0), ('0056', 100.0, 0.0, 50.0), ('0057', -48.0385, -30.0, 50.0), ('0058', -48.0385, 30.0, 50.0),
 ('0059', -100.0, 60.0, 50.0), ('0060', -151.9615, 30.0, 50.0), ('0061', -151.9615, -30.0, 50.0),
 ('0062', -100.0, 0.0, 50.0), ('0063', -48.0385, -30.0, 50.0), ('0064', -48.0385, 30.0, 50.0),
 ('0065', -100.0, 60.0, 50.0), ('0066', -151.9615, 30.0, 50.0), ('0067', -151.9615, -30.0, 50.0),
 ('0068', -100.0, 0.0, 50.0), ('0069', -48.0385, -30.0, 50.0), ('0070', -48.0385, 30.0, 50.0),
 ('0071', -100.0, 60.0, 50.0), ('0072', -151.9615, 30.0, 50.0), ('0073', -151.9615, -30.0, 50.0),
 ('0074', -100.0, 0.0, 50.0), ('0075', -48.0385, -30.0, 50.0), ('0076', -48.0385, 30.0, 50.0),
 ('0077', -100.0, 60.0, 50.0), ('0078', -151.9615, 30.0, 50.0), ('0079', -151.9615, -30.0, 50.0),
 ('0080', -100.0, 0.0, 50.0), ('0081', -48.0385, -30.0, 50.0), ('0082', -48.0385, 30.0, 50.0),
 ('0083', -100.0, 60.0, 50.0), ('0084', -151.9615, 30.0, 50.0), ('0085', -151.9615, -30.0, 50.0),
 ('0086', -100.0, 0.0, 50.0), ('0087', -48.0385, -30.0, 50.0), ('0088', -48.0385, 30.0, 50.0),
 ('0089', -100.0, 60.0, 50.0), ('0090', -151.9615, 30.0, 50.0), ('0091', -151.9615, -30.0, 50.0),
 ('0092', -100.0, 0.0, 50.0), ('0093', -48.0385, -30.0, 50.0), ('0094', -48.0385, 30.0, 50.0),
 ('0095', -100.0, 60.0, 50.0), ('0096', -151.9615, 30.0, 50.0), ('0097', -151.9615, -30.0, 50.0),
 ('0098', -100.0, 0.0, 50.0), ('0099', -48.0385, -30.0, 50.0), ('0100', -48.0385, 30.0, 50.0),
 ('0101', -100.0, 60.0, 50.0), ('0102', -151.9615, 30.0, 50.0), ('0103', -151.9615, -30.0, 50.0),
 ('0104', -100.0, 0.0, 50.0), ('0105', -48.0385, -30.0, 50.0), ('0106', -48.0385, 30.0, 50.0),
 ('0107', -100.0, 60.0, 50.0), ('0108', -151.9615, 30.0, 50.0), ('0109', -151.9615, -30.0, 50.0),
 ('0110', -100.0, 0.0, 50.0), ('0111', -48.0385, -30.0, 50.0), ('0112', -48.0385, 30.0, 50.0),
 ('0113', -100.0, 60.0, 50.0), ('0114', -151.9615, 30.0, 50.0), ('0115', -151.9615, -30.0, 50.0),
 ('0116', -100.0, 0.0, 50.0), ('0117', -48.0385, -30.0, 50.0), ('0118', -48.0385, 30.0, 50.0),
 ('0119', -100.0, 60.0, 50.0), ('0120', -151.9615, 30.0, 50.0), ('0121', -151.9615, -30.0, 50.0),
 ('0122', -100.0, 0.0, 50.0), ('0123', -48.0385, -30.0, 50.0), ('0124', -48.0385, 30.0, 50.0),
 ('0125', -100.0, 60.0, 50.0), ('0126', -151.9615, 30.0, 50.0), ('0127', -151.9615, -30.0, 50.0),
 ('0128', -100.0, 0.0, 50.0), ('0129', -48.0385, -30.0, 50.0), ('0130', -48.0385, 30.0, 50.0),
 ('0131', -100.0, 60.0, 50.0), ('0132', -151.9615, 30.0, 50.0), ('0133', -151.9615, -30.0, 50.0),
 ('0134', -100.0, 0.0, 50.0), ('0135', -48.0385, -30.0, 50.0), ('0136', -48.0385, 30.0, 50.0),
 ('0137', -100.0, 60.0, 50.0), ('0138', -151.9615, 30.0, 50.0), ('0139', -151.9615, -30.0, 50.0),
 ('0140', -100.0, 0.0, 50.0), ('0141', -48.0385, -30.0, 50.0), ('0142', -48.0385, 30.0, 50.0),
 ('0143', -100.0, 60.0, 50.0), ('0144', -151.9615, 30.0, 50.0), ('0145', -151.9615, -30.0, 50.0),
 ('0146', -100.0, 0.0, 50.0), ('0147', -48.0385, -30.0, 50.0), ('0148', -48.0385, 30.0, 50.0),
 ('0149', -100.0, 60.0, 50.0), ('0150', -151.9615, 30.0, 50.0), ('0151', -151.9615, -30.0, 50.0),
 ('0152', -100.0, 0.0, 50.0), ('0153', -48.0385, -30.0, 50.0), ('0154', -4
```

图 2

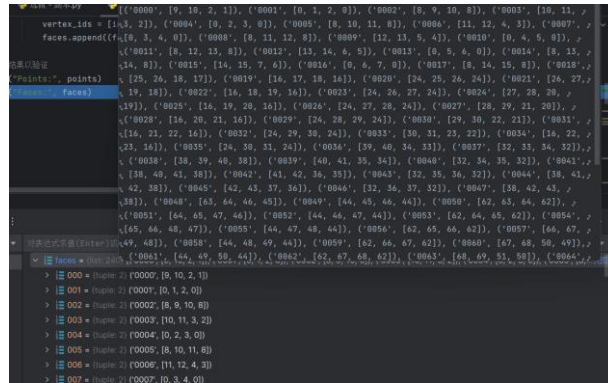


图 3

$$\mathbf{c} = [\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}]$$

然后计算向量 $\mathbf{a}$ 和 $\mathbf{b}$ ：

这里我们使用标准化的单位向量

$$\mathbf{h} = [0,0,1]$$

则

$$\mathbf{a} = \mathbf{h} \times \mathbf{c} = [0,0,1] \times [\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}] = [-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, 0]$$

$$\mathbf{b} = \mathbf{c} \times \mathbf{a} = [\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}] \times [-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, 0] = [-\frac{1}{3}, -\frac{1}{3}, \frac{2}{3}]$$

计算透视变换矩阵 $\mathbf{T}$ ：

$$\mathbf{T} = \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix} = \begin{bmatrix} -\frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & 0 \\ -\frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{3}} & \frac{2}{3} \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} \end{bmatrix}$$

使用代码计算如图 4 所示：

```

110 import numpy as np
111
112 # 视点 V 和物点 D 的坐标
113 V = np.array([500, 500, 500])
114 D = np.array([0, 0, 0])
115
116 # 计算向量 c 并标准化
117 c = V - D
118 c_norm = c / np.linalg.norm(c)
119
120 # 计算向量 a 和 b
121 h = np.array([0, 0, 1])
122 a = np.cross(h, c_norm)
123 b = np.cross(c_norm, a)
124
125 # 构建透视变换矩阵 T
126 T = np.vstack((a, b, c_norm))
127
128 # 打印透视变换矩阵 T
129 print("透视变换矩阵 T:")

```

图 4

得到结果如图 5 所示：

```

[[ -0.57735027  0.57735027  0.          ]
 [ -0.33333333 -0.33333333  0.66666667]
 [ 0.57735027  0.57735027  0.57735027]]

```

进程已结束,退出代码0

图 5

如下代码计算各点透视坐标：

```

# 计算向量 c 并标准化
c = V - D
c_norm = c / np.linalg.norm(c)

# 计算向量 a 和 b
h = np.array([0, 0, 1])
a = np.cross(h, c_norm)
b = np.cross(c_norm, a)

# 构建透视变换矩阵 T
# 构建透视变换矩阵 T (应该是4x4矩阵)

# 提取每个点的后三个元素，即坐标
coordinates = np.array([point[1:] for point in points])

# 计算透视坐标
perspective_coors = []
for point in coordinates:
    # 平移至原点
    translated_point = np.array([point[0] - V[0], point[1] - V[1], point[2] - V[2]])

    # 计算透视坐标
    xc = -c_norm[1] * translated_point[0] + c_norm[0] * translated_point[1]
    yc = -c_norm[0] * translated_point[0] - c_norm[1] * c_norm[2] * translated_point[1] + (c_norm[1] * c_norm[1] + c_norm[0] * c_norm[0]) * translated_point[2]
    zc = c_norm[0] * translated_point[0] + c_norm[1] * translated_point[1] + c_norm[2] * translated_point[2]

    perspective_coors.append((xc, yc, zc))

for coord in perspective_coors:

```

图 6

定义视点和物点：

V 是视点的坐标，这里设置为[500, 500, 500]。

D 是物点的坐标，这里设置为[0, 0, 0]。

计算向量 c 并标准化：

向量 c 是从物点 D 到视点 V 的向量，计算方式为 $c=V-D$ 。c_norm 是向量 c 的单位向量，通过 $c/\text{np.linalg.norm}(c)$ 计算得到。

计算向量 a 和 b:

向量 h 是一个垂直向量, 这里设置为[0, 0, 1]。向量 a 是 h 和 c_norm 的叉积, 表示垂直于 c 的一个向量。向量 b 是 c_norm 和 a 的叉积, 表示另一个垂直于 c 的向量。

构建透视变换矩阵 T:

代码中没有显式构建透视变换矩阵 T, 但是通过计算 a、b 和 c, 我们可以推断出透视变换矩阵的结构。透视变换矩阵 T 通常是一个 4x4 矩阵, 用于将三维点转换为透视坐标。然而, 这里的代码直接使用向量运算来模拟透视变换的效果。

提取坐标并计算透视坐标:

coordinates 是一个包含所有点坐标的 NumPy 数组。对于每个点, 代码计算其相对于视点 V 的平移向量 translated_point。然后, 使用向量 c、a 和 b 来计算透视坐标 xc、yc 和 zc。这里的计算公式是基于透视几何的原理, 其中 xc、yc 和 zc 是透视变换后的坐标。

打印透视坐标:

最后, 代码遍历 perspective_coords 列表并打印每个点的透视坐标。

结果如图 7 所示。

```
C:\Users\86182\anaconda3\envs\YOLov8-Deepsort\python.exe "C:/Program Files/JetBrains/PyCharm Community Edition 2022.3.3/
已连接到 pydev 调试器(内部版本号 223.8836.43)(57.7350269189626, 66.66666666666669, -894.89291724392)
(10.414532830777891, 59.3461666666666704, -882.2134393071121)
(45.05554898215462, 39.3461666666666704, -847.5724231557346)
(92.37604307034013, 46.666666666666686, -860.2519010925425)
(105.05552100714806, 73.987166666666672, -907.572395188728)
(70.41450485577853, 93.987166666666672, -942.2134113321056)
(23.09401076758502, 86.66666666666669, -929.5339333952976)
(10.414532830777891, 59.3461666666666704, -882.2134393071121)
(57.7350269189626, 0.0, -952.6279441628826)
(10.414532830777891, -7.3204999999999981, -939.9484662260747)
(45.05554898215462, -27.320499999999998, -905.3074500746972)
(92.37604307034013, -20.0, -917.9869280115051)
(105.05552100714806, 7.3205000000000038, -965.3074220996906)
(70.41450485577853, 27.3205000000000038, -999.9484382510682)
(23.09401076758502, 20.0, -987.2689601142602)
(10.414532830777891, -7.3204999999999981, -939.9484662260747)
(-57.735026918962575, 5.684341860808026-14, -779.4228634059948)
```

图 7

问题(3):

投影平面坐标计算程序代码如图 8 所示

```
72     perspective_coords.append((xc, yc, zc))
73     d = 1.0 # 投影面与视点的距离
74
75     # 计算投影平面坐标
76     projected_coords = []
77     for xc, yc, zc in perspective_coords:
78         if zc != 0: # 避免除以零
79             x = (xc / zc) * d
80             y = (yc / zc) * d
81         else:
82             x = 0
83             y = 0
84         projected_coords.append((x, y))
85
86     # 打印投影平面坐标
87     for coord in projected_coords:
88         print(coord)
89
```

图 8

结果如图 9 所示。

```
C:\Users\86182\anaconda3\envs\Y0L0v8-Deepsort\python.exe
(-0.06451612903225809, -0.07449680892769366)
(-0.01180500360429403, -0.06726962435901794)
(-0.05315834700520457, -0.04642218834842526)
(-0.10738255033557047, -0.054247676299696394)
(-0.11575442528331638, -0.0815220549451963)
(-0.07473307427901943, -0.09975146345432213)
(-0.024844720496894394, -0.09323668943434951)
(-0.01180500360429403, -0.06726962435901794)
(-0.06060606060606063, -0.0)
(-0.01107989768055242, 0.007788192930823154)
(-0.04976822954283329, 0.030178145554580114)
(-0.10062893081761005, 0.02178680261092927)
(-0.1088311543058856, -0.007583594440905506)
```

图 9

问题(4):

设备坐标计算程序代码如图 10 所示。

```
projected_coords.append((x, y))

width, height = 800, 600
scale = 500

# 计算设备坐标
device_coords = []
for x, y in projected_coords:
    xs = (x * scale / width) + (width / 2)
    ys = height - (y * scale / height)
    device_coords.append((xs, ys))

# 打印设备坐标
for coord in device_coords:
    print(coord)
```

图 10

结果如图 11 所示。

```
C:\Users\86182\anaconda3\envs\Y0L0v8-Deepsort\python.exe "C:\U
(399.9596774193548, 600.0620806741064)
(399.9926218727473, 600.0560580202991)
(399.96677603312173, 600.0386851569571)
(399.93288590604027, 600.0452063969165)
(399.92765348419795, 600.0679350457876)
(399.95329182857563, 600.0831262195453)
(399.98447204968943, 600.0776972411953)
(399.9926218727473, 600.0560580202991)
(399.9621212121212, 600.0)
(399.9930750639497, 599.9935098392243)
(399.9688948565357, 599.9748515453712)
(399.937106918239, 599.9818443311575)
(399.9319805285588, 600.0063196620341)
(399.9559886651638, 600.0227682573045)
(399.98538011695905, 600.0168815868184)
(399.9930750639497, 599.9935098392243)
(400.0462962962963, 600.0)
(400.0854745185023, 500.0892677709505)
```

图 11

问题(5):

网格线透视图的绘制，这里通过绘制出每个点对应的设备坐标，并按照 faces 列表里形成每个面所需要的四个点连接得到。

主要代码如图 12 所示。

```
device_coords = []
for x, y in projected_coords:
    xs = (x * scale / width) + (width / 2)
    ys = height - (y * scale / height)
    device_coords.append((xs, ys))
# 创建绘图窗口
plt.figure(figsize=(10, 8))

# 绘制所有设备坐标点
for point in device_coords:
    plt.plot(point[0], point[1], 'o') # 'o' 表示绘制一个圆圈标记点

# 绘制网格线
for face in faces:
    face_vertices = [device_coords[point_id - 1] for point_id in face[1]]
    # 绘制形成面的四边形的边
    for i in range(4):
        start_vertex = face_vertices[i]
        end_vertex = face_vertices[(i + 1) % 4]
        plt.plot([start_vertex[0], end_vertex[0]], [start_vertex[1], end_vertex[1]], 'k-') # 'k-' 表示黑色实线
```

图 12

图片运行结果如图 13 所示。

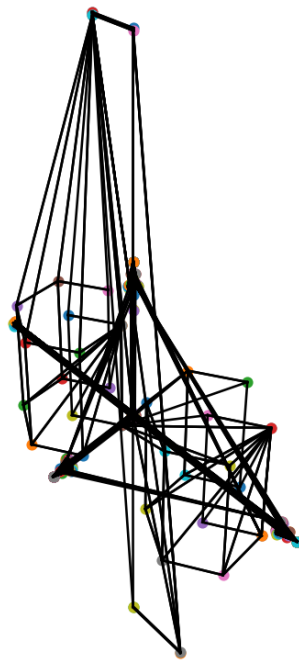


图 13

要计算视点 V 所在的矢量，我们需要利用透视投影的原理。在透视投影中，立方体的每个顶点在屏幕上的投影位置与视点 V 的位置有关。由于立方体的中心位于坐标原点，我们可以假设物点 D 位于原点 (0, 0, 0)。

透视投影的基本公式是：

$$P' = \frac{P-D}{V-D}$$

其中，P 是立方体上的点，P' 是该点在屏幕上的投影，D 是物点（在本题中为原点），V 是视点。

由于我们知道立方体的边长和中心位置，我们可以推断出立方体顶点的坐标。立方体的边长为 100，中心位于原点，那么顶点的坐标可以是：

- (50, 50, 50) (顶点 1)
- (-50, 50, 50) (顶点 2)
- (-50, -50, 50) (顶点 3)
- (50, -50, 50) (顶点 4)
- (50, 50, -50) (顶点 5)
- (-50, 50, -50) (顶点 6)
- (-50, -50, -50) (顶点 7)
- (50, -50, -50) (顶点 8)

然后，我们可以使用这些顶点的屏幕坐标来估计视点 V 的位置。由于透视投影的性质，我们知道透视投影点 P'、物点 D 和视点 V 共线。因此，我们可以通过观察立方体面上的点在透视投影平面上的投影来估计视点 V 的方向。

通过以下步骤来估计视点 V 的矢量：

1. 确定物点 D 的位置：由于立方体的中心为坐标原点，我们可以假设物点 D 位于原点 (0, 0, 0)。

2. 观察透视投影点 P' 的位置：根据题目中给出的屏幕坐标，我们可以观察到立方体面上的点在透视投影平面上的投影。

3. 估计视点 V 的方向：由于透视投影点 P'、物点 D 和视点 V 共线，我们可以通过观察透视投影点 P' 的位置来估计视点 V 的方向。具体来说，我们可以假设视点 V 位于透视投影点 P' 的延长线上，并且远离物点 D。

4. 计算视点 V 的坐标：由于我们只知道透视投影点 P' 的位置和物点 D 的位置，我们无法直接计算出视点 V 的确切坐标。但是，我们可以通过假设视点 V 位于透视投影点 P' 的延长线上，并且远离物点 D，来估计视点 V 的大致位置。

然而，在没有额外信息的情况下，我们无法精确计算出视点 V 所在的矢量。

透视投影涉及到视点 V、物点 D 以及透视投影点 P 之间的关系。要计算视点 V 的矢量，我们需要知道至少以下信息之一：

1. 视点 V 到投影平面的距离 (d)。
2. 透视投影点 P 在投影平面上的确切坐标。
3. 视点 V 的确切坐标或者其相对于物点 D 的方向和距离。

在问题中，我们只知道立方体的边长和中心位置，以及物点 D 位于原点。没有视点 V 的确切坐标，也没有透视投影点 P 的确切坐标，因此无法直接计算出视点 V 的矢量。